Project Report CS-476

Hinary Antoine 324960
Nassif Marc 327323

October 30, 2026

Contents

1 Project Outline **2**
1.1 Project Details 2
1.2 Initial Profiling 2

2 Optimizations **3**
2.1 Sobel Optimizations 3
2.1.1 Software Sobel Optimization 3
2.1.2 Sobel Custom Instruction 3
2.1.3 DMA Implementation 3
2.2 Movement Detection Optimizations 4
2.2.1 Word indexing 4
2.2.2 Movement Detection Custom Instruction 4

3 Results **5**

4 Conclusion **6**

5 Further work **6**

1 Project Outline

1.1 Project Details

The project objective is achieving a real-time movement detection, based on Sobel Filtering for edge detection, and finding differences in detected edges to deduce movement. In real time, this information is written to HDMI-out in the form of a full image.
The basic algorithm to use, provided by the course for edge detection, is to apply the vertical and horizontal Sobel kernels to an image successively, summing their absolute values, and comparing that output to a preset threshold to determine whether an edge exists at that location.
The Sobel kernels are:

$$G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} G_y = \begin{pmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{pmatrix}$$

We simply compare, at every u a 3×3 neighborhood around pixel (x, y), $|G_x u| + |G_y u| > 128$. This first task is accomplished by our `edgeDetection` functions.

Afterwards, we simply save that binary output in a table. Once we have the result for two successive images, we can simply compute whether a movement occurred at a pixel location by checking $u_1 \oplus u_2$, where $\oplus$ is the exclusive-or, or XOR, operator, which checks for bit-wise inequality.
Finally, every bit produced must be decoded into either 0 or 255 for image output. These last two tasks are accomplished by our various `detect_moves` functions.

1.2 Initial Profiling

A first profiling reveals that a full 195 million cycles are spent just for computing the Sobel filter, and we spend 17 million cycles on movement detection. Furthermore, we have a very large CPU stall time, at a full 170 million cycles. The main culprit, and main target for optimisation, taking 90% of execution time, is clearly the Sobel filtering: an initial hypothesis is that the large number of memory accesses needed for computing this filter make it overly dependent on waiting on bus accesses for reading small data bits, and needing to jump between different lines of data sporadically does not help for caching either.

2 Optimizations

2.1 Sobel Optimizations

2.1.1 Software Sobel Optimization

An initial software optimization made was employing separable filtering.
The Sobel filter is a separable filter, meaning its output can be computed as the result of successive single-dimensional filters. Namely:

$$G_x = \begin{pmatrix}1\\2\\1\end{pmatrix} * \begin{pmatrix}1 & 0 & -1\end{pmatrix} G_y = \begin{pmatrix}1\\0\\-1\end{pmatrix} * \begin{pmatrix}1 & 2 & 1\end{pmatrix}$$

This is advantageous, as it means we can compute the horizontal G_x and G_y components as a line of the image is read in, and memorize that output to a buffer. Once a third line is read in, we can use each pixel to apply the vertical components. This means we generally have more consecutive usage of data, as the image itself is only ever read line-by-line, which is advantageous for caching purposes, and the vertical filtering is applied to separate buffers. It also means each pixel is read fewer times, hence decreasing memory usage.
This solution is implemented in the `edgeDetectionSep` function. The preprocessor statement:

```
#define __DMA_SOBEL__
```

must be commented out for proper execution.

2.1.2 Sobel Custom Instruction

An alternate solution is to use a Custom Instruction to compute the full, non-separate Sobel filter in a single-cycle[1] for each pixel. This was implemented in the `sobelCi.v` module, using minimal adders to produce the thresholded result of the kernel sums immediately. However, this implementation on its own does not offer an advantage over the software optimization - we have more memory to access, and we cannot take advantage of separability, due to needing larger buffers in FPGA for full advantage.

2.1.3 DMA Implementation

We now use a DMA module to read in camera lines one-by-one. Once the third line starts entering, we may compute our Sobel thresholded output using our Custom Instruction, and overwrite it onto the first line's previous pixel. By doing this computation sequentially over the entire line, then using a DMA write operation to write it out, we can implement a sort of three-way ping-pong, juggling three lines of pixels, while only ever reading in or writing out one of them at any given time. The implementation of this DMA gives us two guarantees:

- We can access every single pixel in two cycles (the time of a read operation) independently of bus occupancy.

[1] Not quite, due to memory access time

- We can compute output values while the DMA is reading in a new line.

This yields very large gains in performance, as the parallelism obtained in memory management and computation results in computing times much faster than previous methods.

One particularity of this method that bears mentioning: We read in buffer values as 32-bit words, however, each pixel is only 8-bits wide. Hence, we actually get 4-pixels for each word read out of the DMA buffer. To ensure proper function, we actually need to "look ahead" and compute the Sobel filter values centered at the 2nd, 3rd and 4th pixel in our current middle-buffer, as well as the Sobel filter centered at the 1st pixel of the next buffer. In our application, in `edgeDetectionDma`, we do this by treating our 32-bit word as an array of bytes, allowing for more natural accessing of variables.

2.2 Movement Detection Optimizations

2.2.1 Word indexing

The translation for bit-differences to byte-outs for the HDMI-out are a source of complexity in this project: though the single-line difference detection with XOR may be simple and natural, writing out the result must be done by inspecting the resulting binary output, and, for every single bit, writing out a byte of 0 or 255 to the result array. This is inefficient, specifically since we have a 32-bit bus, meaning ideally, data should be sent and received in words and not bytes to maximize efficiency. The `detect_moves_words` function does just that, reading the Sobel results in word-by-word, and writing out 4-pixels at a time, hence a 32-bit word every time. However, the required data manipulation (shifting, logical-ors, checking each bit individually) leaves room for improvement, specifically hardware acceleration.
To use this optimization, make sure to enable the preprocessor directive:

```
#define __WORD_MVMT__
```

2.2.2 Movement Detection Custom Instruction

Since our decoding is slowed down by shifting and or-ing, and we have relatively compact data to work with (32-bits of differences provide us a full 32 pixels for the output), we can employ a Custom Instruction, which takes in the 32-bit difference and an index, to produce out 4 pixels at once, which can be immediately written out to our HDMI-out. This is accomplished by the `moveCi` module, and employed by the `detect_moves_words_ci`.
To use this optimization, make sure to enable the preprocessor directive:

```
#define __MVMT_CI__
```

3 Results

DMA Sobel	Move Ci	Word move	Total Cycles	Cycles Sobel	Cycles Move	Bus idle	CPU stall
Yes	Yes	Yes	14.1M	10.7M	3.4M	10.0M	3.5M
Yes	NO	Yes	18.6M	10.7M	7.9M	13.8M	4M
Yes	NO	NO	26.5M	10.7M	15.8M	17.5M	10.8M
NO	Yes	Yes	199M	195M	3.6M	87.3M	162M
NO	NO	Yes	204M	195M	7.9M	91.1M	163M
NO	NO	NO	212M	195M	17M	94.8M	170M

Table 1: Performance metrics with different optimizations

As this table illustrates, we were able to go down from an initial runtime of 212 million cycles (so about 3 seconds per update on our 74 MHz clock) down to 14.1 million cycles. Note that it does not make sense to use the Move Ci without Word indexing for our movement detection. Note as well the bottom-most implementation is using the separable Sobel Filter, as described in §2.1.1

These results can be replicated simply by changing the `#define` directives in `project.c`.
A result not included here is using the Sobel filter with just `edgeDetection`, as this implementation is even slower, at about 350 million cycles just to compute the Sobel filter.
Notice the dramatic improvement obtained by using a DMA for our Sobel computation, a full order of magnitude of optimization, with almost every saved cycle also decreasing the CPU stall cycles, indicating we spend less time idly waiting to read data in, thanks to the high efficiency of data transfers enabled by using DMAs.

As for our movement detection optimisation, we see substantial improvement, in total decreasing its runtime by 80%. Its optimization with word indexing does bring down CPU stall cycles substantially, and the use of the Custom Instruction improves Bus idle cycles, as we spend less time on computations with present data, meaning greater throughput and bus utilization.

4 Conclusion

Using DMAs to simultaneously read pixels in and treating data, as well as employing smarter data packing and indexing, aided by custom instruction accelerators, we are able to move from a painfully slow, roughly 0.3 fps performance, up to a much quicker 5 fps, getting a responsive video feed out of our data.

The crux of the optimisations in our work rely on avoiding needless computing when it can be performed by a Ci in a much quicker fashion, and maximizing bus use whenever possible, especially in the context of our project, where it is all about data transfer and treatment. Indeed, every bus idle cycle is an opportunity for optimization, the ideal project having a fully occupied bus, as data constantly is traded between the camera and the output.

Many thanks to Prof. Theo Kluter and TA Canberk Sönmez for their support throughout the semester, and for offering this highly interesting, specialized course into a subject often glossed over. This course and project offered great insight into efficient computing and system design and prototyping.

5 Further work

Avenues for improvement would probably be implementing a more efficient edge detection. After computing the Sobel filter for a line, the previously computed Sobel output for that line could be read in and immediately XOR-ed in circuit, allowing for a direct output of the differences - thus the movement. This would require large restructuring, including changing the program structure in the edge detection section to allow for a more dynamic in-and-out reading, as well as probably incorporating larger buffers into the ramDma.

Other improvements could be centered around the movement detection and decoding section of the code, an option being some custom cache memory where two inputs can be provided and XOR-ed at once, allowing for direct writing of the new Sobel filter and comparing it to the previous one, with a DMA write-out to memory.

Appendix

Find a demonstration at: https://youtu.be/5rFBaMkaR-A